**Desarrollo de software para móviles**

**DSM941**

**Ciclo 02-2026**

**G01T**




# FORO 1

**Docente: Alexander Alberto Siguenza Campos**

**Integrantes del equipo:**

| Apellidos | Nombres | Carné |
|---|---|---|
| **Berdugo Artiga** | **Lisandro Rafael** | **BA131462** |
| | | |
| | | |
| | | |

Link video: Video foro 1

Link GitHub: https://github.com/LisandroBerdugo/ImportSV.git

21/10/2026

**¿Qué es Jetpack Compose?**

Jetpack Compose es un kit de herramientas para crear interfaces nativas de Android a través de funciones de Kotlin marcadas con la anotación @Composable. Estas funciones describen qué elementos deben mostrarse, cómo se ordenan y qué valores utilizan. Compose trabaja junto con las bibliotecas del ecosistema Android y puede coexistir con vistas tradicionales durante procesos de modernización (Android Developers, s. f.-a).

Su principal diferencia conceptual es el paso de un modelo imperativo a uno declarativo. En un diseño imperativo, la aplicación suele buscar componentes existentes y modificar sus propiedades. En el declarativo, las funciones describen el resultado visual deseado para el estado actual; cuando el estado cambia, Compose actualiza la parte de la interfaz que lo requiere.Composición, recomposición y estado

La composición es la estructura de elementos componibles que representa la interfaz. La composición inicial aparece cuando se ejecutan las funciones correspondientes por primera vez. La recomposición ocurre cuando cambian los valores de estado observados por la interfaz, permitiendo actualizar los elementos afectados. La documentación de Android destaca que la actualización del estado no implica necesariamente reconstruir toda la pantalla (Android Developers, s. f.-b).

Las funciones remember y mutableStateOf ayudan a mantener valores observables durante las recomposiciones, mientras que rememberSaveable permite conservar determinados estados compatibles ante recreaciones de la actividad. Para interfaces complejas, es aconsejable elevar el estado hacia el componente responsable y exponer los datos junto con funciones de respuesta a eventos; este patrón se conoce como state hoisting (Android Developers, s. f.-b).

### Flujo unidireccional de datos

Una buena práctica consiste en enviar el estado desde el nivel superior hacia los componentes visuales y comunicar las acciones del usuario en sentido contrario. Así, los componentes reciben los valores que deben representar y notifican eventos sin apropiarse de las decisiones de negocio. El enfoque facilita las pruebas y reduce la duplicación de estados.

### Componentes para la construcción de interfaces

Compose ofrece elementos de organización como Column, Row y Box, que distribuyen contenido verticalmente, horizontalmente o de forma superpuesta. El modificador Modifier permite establecer relleno, tamaño, alineación, comportamiento de desplazamiento y otras características visuales. Los componentes Text, Button, TextField, Card y Scaffold cubren necesidades frecuentes, mientras que Material 3 proporciona estilos y pautas de diseño consistentes (Android Developers, s. f.-c).

Una pantalla puede construirse a partir de componentes pequeños y reutilizables. Por ejemplo, un campo de entrada con etiqueta y mensaje de error puede convertirse en un componente común, evitando replicar código en varios formularios. Esta reutilización también mejora la coherencia estética y la facilidad de mantenimiento.

### Navegación entre pantallas

Las aplicaciones con múltiples pantallas necesitan definir destinos y controlar los cambios entre ellos. Navigation Compose ofrece mecanismos para establecer un grafo de navegación y desplazarse entre destinos componibles. La separación de rutas, argumentos y acciones de navegación ayuda a evitar dependencias innecesarias entre pantallas (Android Developers, s. f.-d).

En aplicaciones pequeñas también se puede alternar entre pantallas mediante estado; sin embargo, cuando el número de destinos aumenta, una solución de navegación estructurada facilita la gestión del botón Atrás, los argumentos, el historial y los enlaces profundos. La selección depende del alcance y de los requisitos de navegación.

### Formularios y validaciones

Los formularios combinan campos de texto, selecciones, botones y mensajes de error. En Compose, los valores de un formulario forman parte del estado de la interfaz y deben actualizarse mediante eventos. Las validaciones pueden comprobar obligatoriedad, longitud, formato y coherencia entre campos. Su resultado puede presentarse junto al componente correspondiente para ofrecer retroalimentación inmediata al usuario.

Es conveniente diferenciar validaciones de interfaz y reglas de negocio. Las primeras mejoran la experiencia del usuario; las segundas deben mantenerse en una capa apropiada y ejecutarse aunque cambie la interfaz. Asimismo, una validación visual no sustituye la verificación en capas de datos o servicios cuando la aplicación interactúa con sistemas externos.

### Arquitectura y organización del código

Aunque Compose se centra en la capa de presentación, su uso adecuado implica integrarlo en una arquitectura con responsabilidades delimitadas. Las pantallas muestran información y emiten eventos; los gestores de estado o ViewModel coordinan los datos necesarios; las capas inferiores resuelven reglas de negocio y acceso a información. La documentación de arquitectura Android recomienda mantener una fuente de verdad coherente y utilizar flujos de datos observables según las necesidades de la aplicación (Android Developers, s. f.-e).

Separar componentes visuales, lógica y almacenamiento mejora la capacidad de prueba, facilita el reemplazo de tecnologías y reduce el acoplamiento. Compose no exige un patrón arquitectónico único, pero se beneficia de una clara delimitación entre presentación y procesamiento.

## Accesibilidad, rendimiento y pruebas

El diseño accesible exige etiquetas comprensibles, contraste adecuado, áreas táctiles suficientes y descripciones semánticas para los servicios de accesibilidad. Compose incorpora herramientas de semántica y pruebas que permiten comprobar cómo se identifican y utilizan los elementos de la interfaz (Android Developers, s. f.-f).

En cuanto al rendimiento, debe evitarse realizar cálculos costosos de forma repetida durante las recomposiciones. También resulta útil utilizar componentes de listas eficientes cuando se muestran muchos registros, mantener estados en el nivel correcto y evaluar el comportamiento en dispositivos reales. Las pruebas deberían abarcar visualización, interacción, navegación, mensajes de error y comportamiento ante cambios de configuración.

## Ventajas y limitaciones

### Ventajas

Entre sus ventajas destacan la sintaxis declarativa, la posibilidad de reutilizar componentes, la integración con Kotlin, la actualización reactiva de la interfaz y la disponibilidad de herramientas de vista previa. Material 3 aporta elementos coherentes para el diseño y Compose se integra con las capacidades de Android.

## Limitaciones y retos

Entre los retos se encuentran la curva de aprendizaje del enfoque declarativo, los errores derivados de una administración inadecuada del estado, la necesidad de conocer los ciclos de vida de Android y el mantenimiento de bibliotecas compatibles. La adopción de Compose no elimina la necesidad de diseño, arquitectura, seguridad, accesibilidad ni pruebas de calidad.

**Referencias**

Android Developers. (s. f.-a). Jetpack Compose.

https://developer.android.com/develop/ui/compose

Android Developers. (s. f.-b). Estado y Jetpack Compose.

https://developer.android.com/develop/ui/compose/state?hl=es-419

Android Developers. (s. f.-c). Material Design 3 en Compose.

https://developer.android.com/develop/ui/compose/designsystems/material3

Android Developers. (s. f.-d). Navigation with Compose.

https://developer.android.com/develop/ui/compose/navigation

Android Developers. (s. f.-e). Guide to app architecture.

https://developer.android.com/topic/architecture

Android Developers. (s. f.-f). Accessibility in Compose.

https://developer.android.com/develop/ui/compose/accessibility